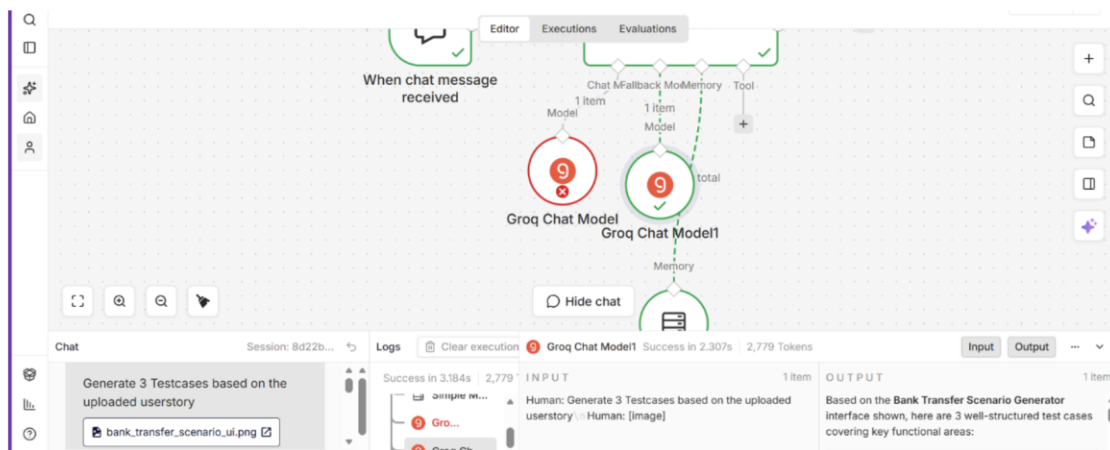
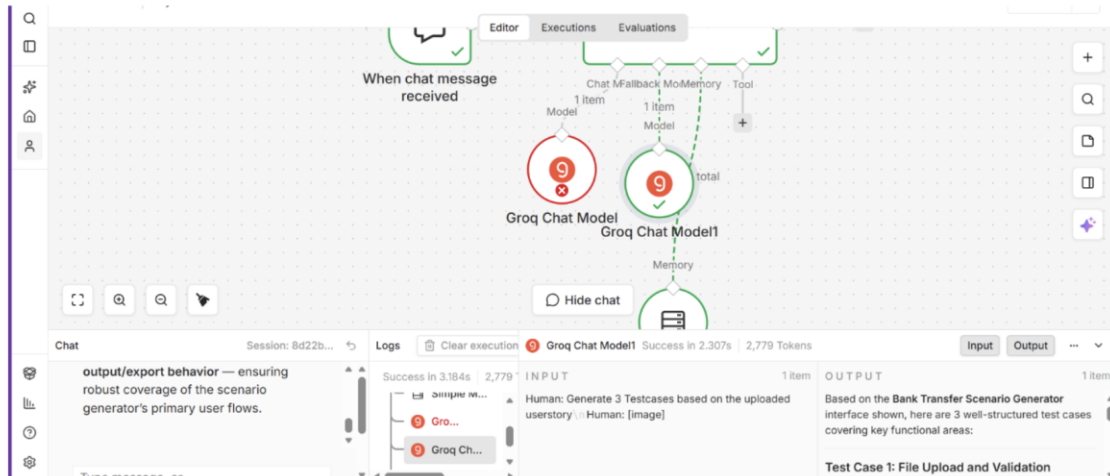


Week-5 [Day2]

Class Assignment

1. Fall back and Memory:



Expected Results: ☒ CSV export contains all 25 scenarios with accurate data and proper formatting.
☒ Gherkin export produces valid BDD-style test scripts with logical Given/When/Then structure.
☒ No data loss or corruption during export.

These test cases cover **input validation, core functionality, and output/export behavior** — ensuring robust coverage of the scenario generator's primary user flows.

Type message, or press 'up' for previous one

When chat ...

AI Agent

Simple...

G...

Groq...

Simple...

Human: Generate 3 Testcases based on the uploaded userstory Human: [image]

Test Case 3: Export Functionality with Different Formats

Objective: Verify that generated scenarios can be exported in CSV and Gherkin formats with correct content and formatting.

Preconditions:
25 scenarios have been successfully generated (as in Test Case 2).
User is on the "Export" tab.

Steps:
1. In "Export" tab, select "CSV" as output format.
2. Click "Download" or equivalent export button.
3. Open downloaded CSV file:
Confirm header row includes relevant columns (e.g., scenarioId, transferType, amount, edgeCase, fromAccount, toAccount).
Verify 25 data rows match generated scenarios.
Check for proper quoting/escaping of special characters.
4. Repeat steps 1-3 for "Gherkin" format.
Confirm output follows Gherkin syntax (Feature:, Scenario:, Given, when, Then).
Verify each scenario maps to a readable test step (e.g., Given a domestic NEFT transfer of ₹2,500).

Expected Results:

Expected Results: ☒ CSV export contains all 25 scenarios with accurate data and proper formatting.
☒ Gherkin export produces valid BDD-style test scripts with logical Given/When/Then structure.
☒ No data loss or corruption during export.

These test cases cover **input validation, core functionality, and output/export behavior** — ensuring robust coverage of the scenario generator's primary user flows.

Type message, or press 'up' for previous one

When chat ...

AI Agent

Simple...

G...

Groq...

Simple...

Human: Generate 3 Testcases based on the uploaded userstory Human: [image]

Steps:
1. In "Export" tab, select "CSV" as output format.
2. Click "Download" or equivalent export button.
3. Open downloaded CSV file:
Confirm header row includes relevant columns (e.g., scenarioId, transferType, amount, edgeCase, fromAccount, toAccount).
Verify 25 data rows match generated scenarios.
Check for proper quoting/escaping of special characters.
4. Repeat steps 1-3 for "Gherkin" format.
Confirm output follows Gherkin syntax (Feature:, Scenario:, Given, when, Then).
Verify each scenario maps to a readable test step (e.g., Given a domestic NEFT transfer of ₹2,500).

Expected Results:
☒ CSV export contains all 25 scenarios with accurate data and proper formatting.
☒ Gherkin export produces valid BDD-style test scripts with logical Given/When/Then structure.
☒ No data loss or corruption during export.

These test cases cover **input validation, core functionality, and output/export behavior** — ensuring robust coverage of the scenario generator's primary user flows.

Editor Executions Evaluations

When chat message received

Chat Memory Memory Tool

Model 1 Item

Groq Chat Model

Groq Chat Model1

total

Hide chat

Chat Session: 8d22b... Logs Clear execution Groq Chat Model Success in 4.488s 2,667 Tokens

Generate another 2 Testcases for the previously uploaded userstory

Type message, or

Success in 7.331s 2,667 Tokens

Human: Generate 3 Testcases based on the uploaded userstory AI: Based on the **Bank Transfer Scenario Generator** interface shown, here are 3 well-structured test cases covering key functional areas:
Test Case 1: File Upload and Validation
Objective: Verify that valid files

Test Case 4 – Pre-generation Validation – Required Fields & Logical Constraints

Test-Case ID TC-04-REQ-VAL

Verify that the Generate button is disabled (or shows a validation error)

052e374611f
(click to copy)

3. Execute browsers (Chrome ≥ 116, Edge ≥ 116, Firefox ≥ 115) to verify cross-browser consistency.

4. Record actual timings for TC-05-PERF-LARGE; if they exceed the SLA, raise a performance ticket with the dev team.

These two cases round out the coverage by testing input validation logic and system scalability, two critical aspects that were not addressed in the first three test cases.

Type message, or press 'up' for previous one

When chat ...

AI Agent

Grog ...

Simple...

Human: Generate 3 Testcases based on the uploaded userstory

AI: Based on the Bank Transfer Scenario Generator interface shown, here are 3 well-structured test cases covering key functional areas:

Test Case 1: File Upload and Validation

Objective: Verify that valid files (CSV/XLSX/JSON) can be uploaded and invalid ones are rejected with appropriate feedback.

Preconditions: User is on the "Attach data" tab. Browser supports drag-and-drop and file selection.

Steps: 1. Drag and drop a valid 'accounts_master.csv' file into the drop zone. 2. Observe the file appears in "Attached files" list with size displayed. 3. Click "+ Add another file" and browse to select 'sample_txns.json'. 4. Confirm both files appear correctly in the list. 5. Attempt to upload an unsupported file type (e.g., '.pdf'). 6. Observe error message appears: "Only CSV, XLSX or JSON files are supported."

Expected Results: Valid files upload successfully and appear in list with correct names/sizes. Invalid file type triggers clear error message; file not added to list.

Below are two additional, independent test cases that complement the three already supplied.

Both are written in a standard "Given-When-Then" style and include all the artefacts you'll need to execute them (ID, title, pre-conditions, steps, expected results, post-conditions).

Test Case 4 – Pre-generation Validation – Required Fields & Logical Constraints

Test-Case ID: TC-04-REQ-VAL

Title: Verify that the Generate button is disabled (or shows a validation error) when mandatory inputs are missing or logically inconsistent.

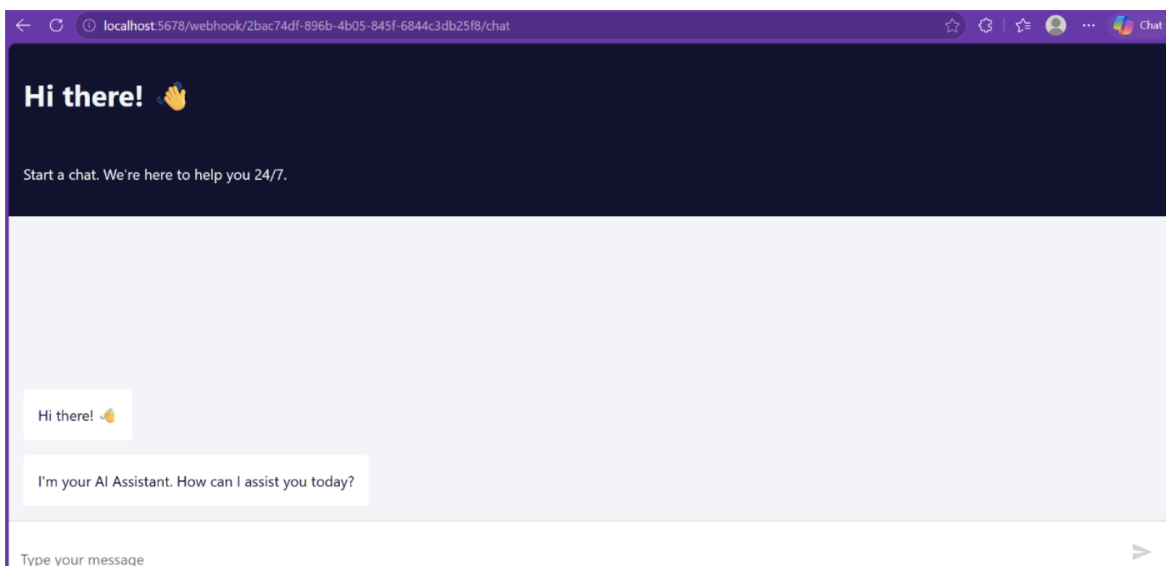
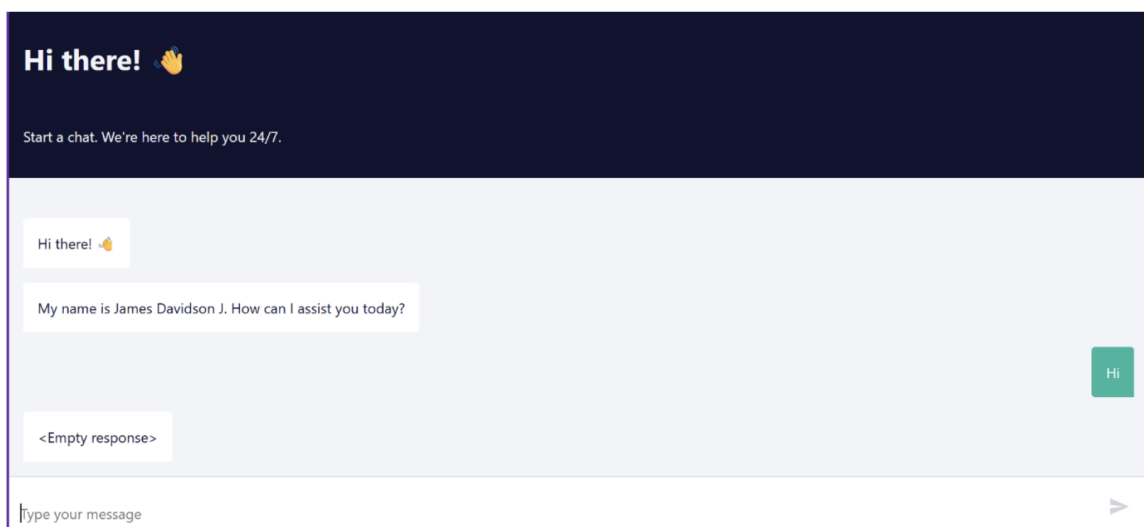
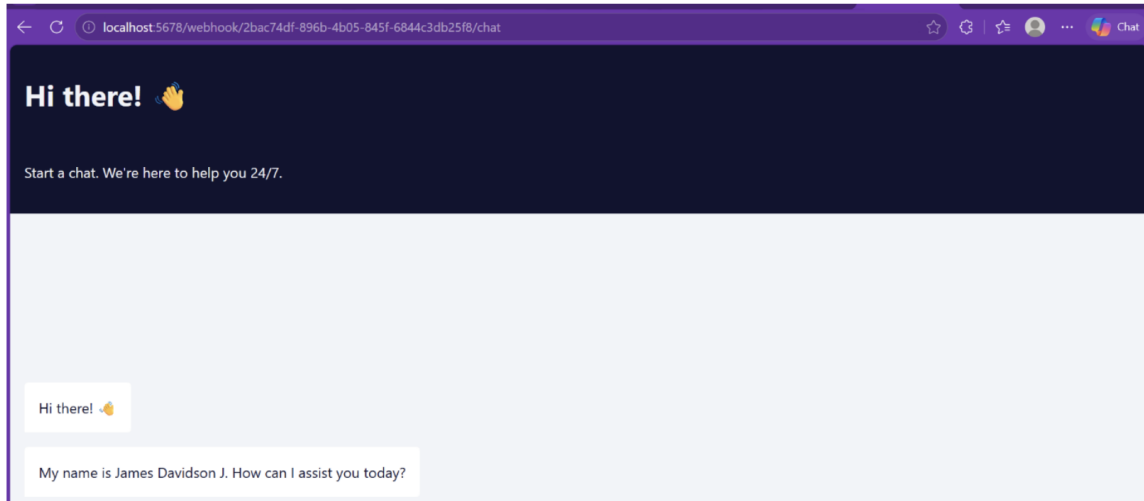
Objective: Ensure the UI prevents scenario generation until the user supplies a complete, valid configuration, and that clear error messages are shown for each missing/invalid field.

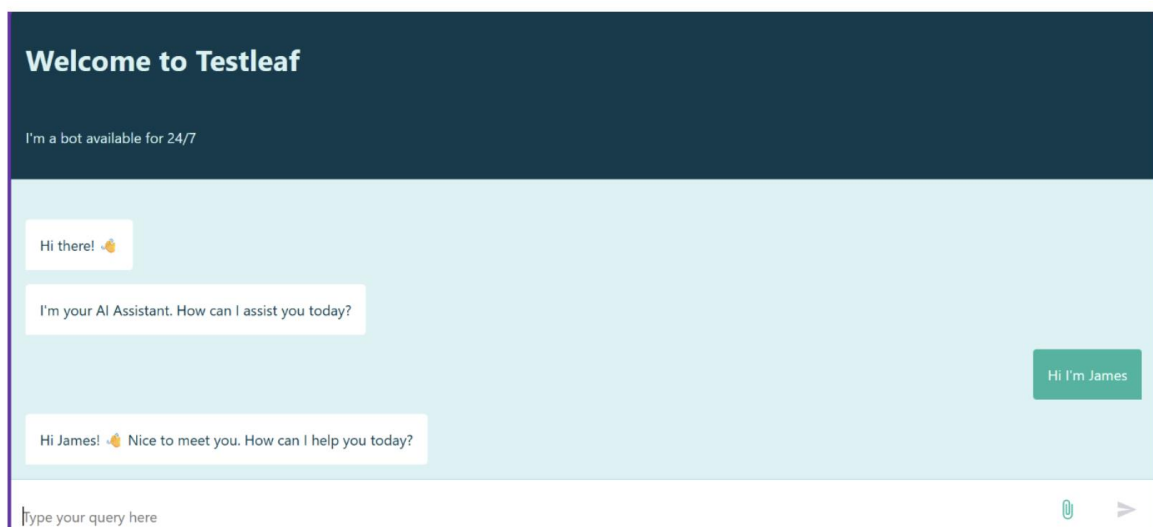
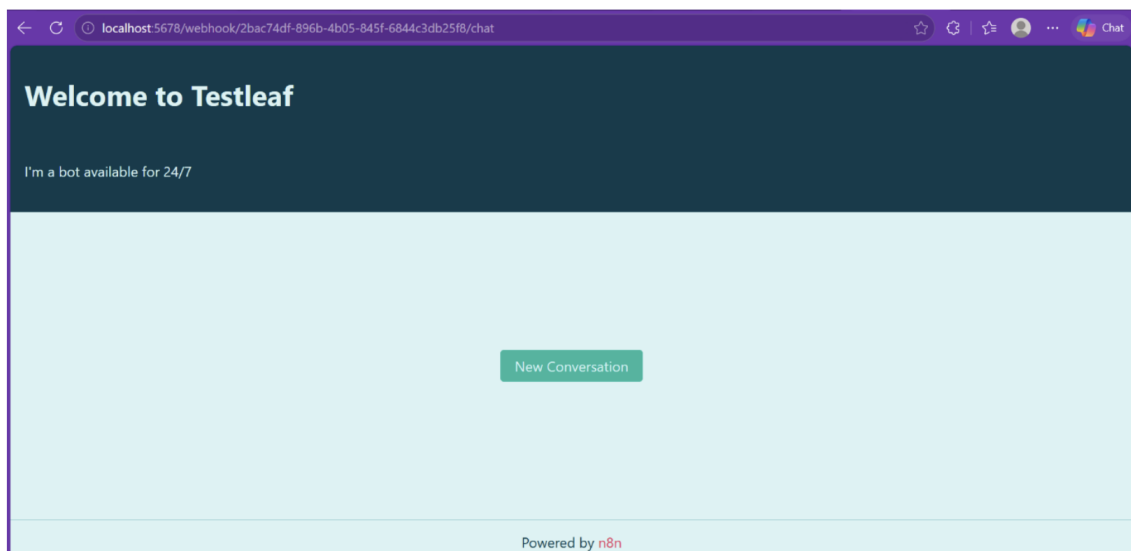
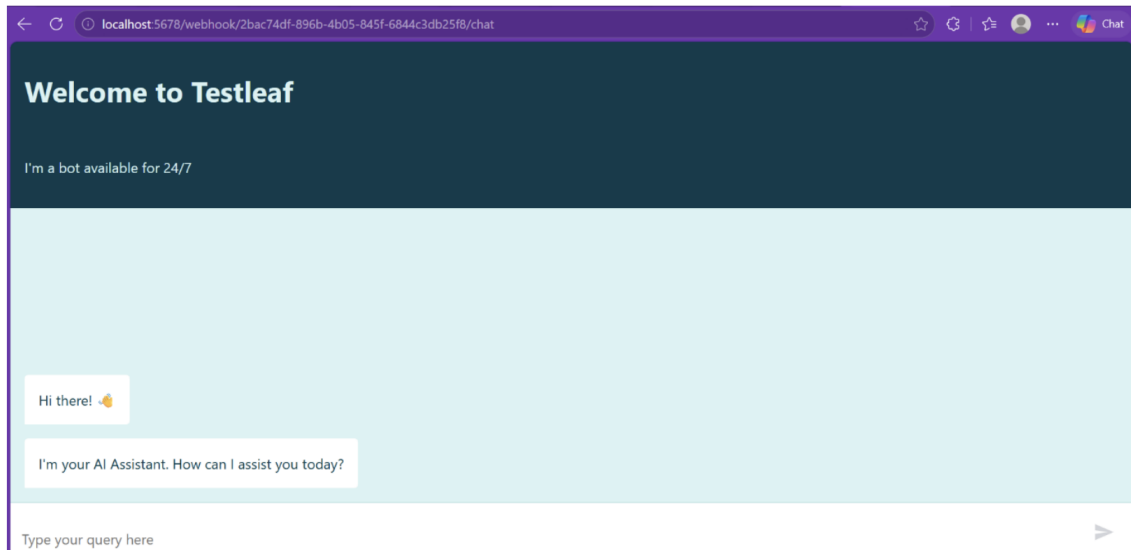
Pre-conditions: 1. User is on the Generate tab. 2. At least one source file has already been attached (e.g., accounts_master.csv).

Test Data: • Transfer-type check-boxes: none selected. • Amount range: Min = 0, Max = 0 (invalid). • Scenario-count: -5 (negative). • Edge-case check-boxes: none selected.

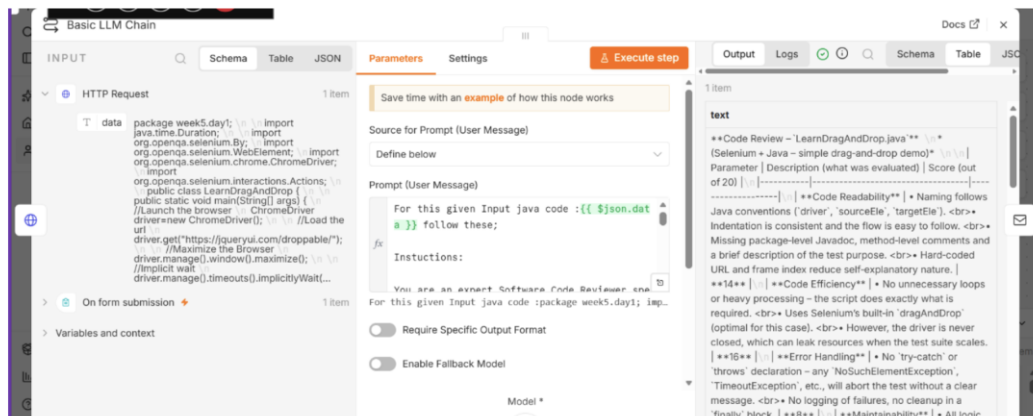
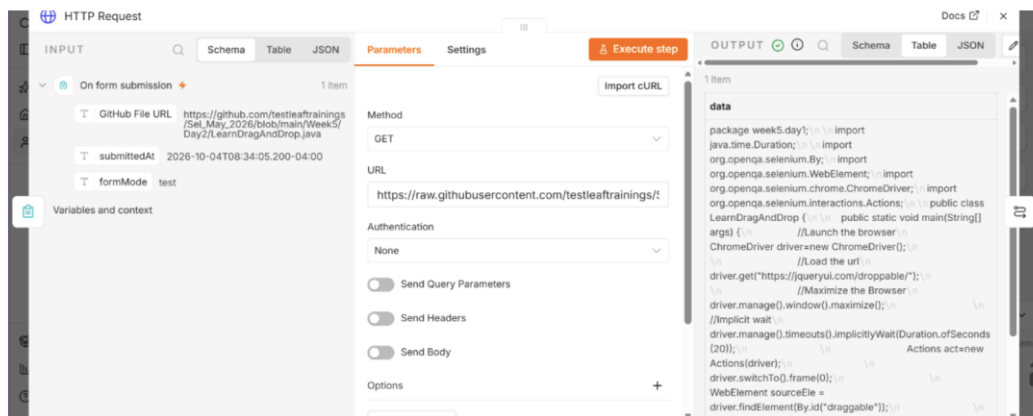
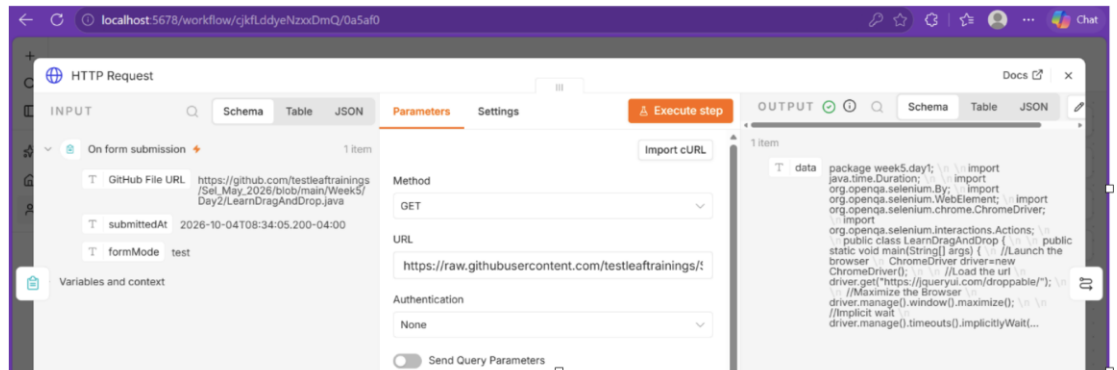
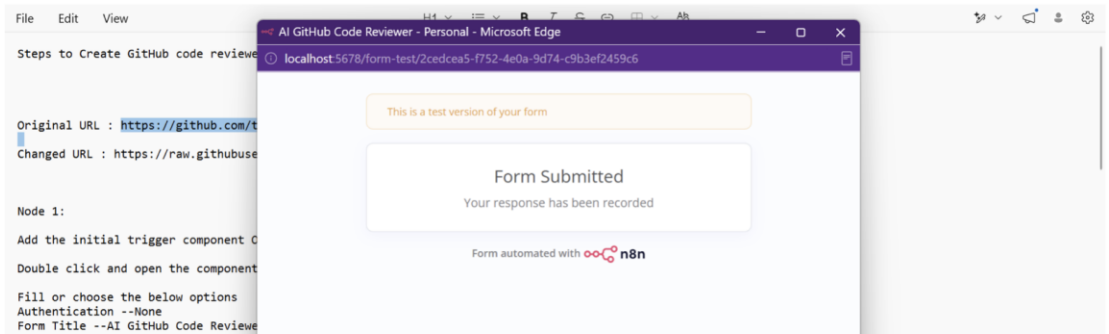
1. Verify the Generate button is disabled (greyed out) immediately after file upload. 2. Attempt to click the button – confirm no action occurs. 3. Select no transfer-type checkbox. 4. Observe

2. In-built UI-n8n:





3. AI GitHub Code Reviewer:



Send an Email

INPUT

Basic LLM Chain

text

```
Code Review - 'LearnDragAndDrop.java'
(Selenium + Java - simple drag-and-drop demo)
Parameter | Description (what was evaluated) | Score (out of 20)
-----|-----|-----
Code Readability | Naming follows Java conventions (driver, sourceFile, targetFile). <br> Indentation is consistent and the flow is easy to follow. <br> Missing package-level Javadoc, method-level comments and a brief description of the test purpose. <br> Hard-coded URL and frame index reduce self-explanatory nature. | 14
Code Efficiency
```

HTTP Request

On form submission

Variables and context

Parameters

Credential

SMTP account

Operation

Send

From Email

jameschristster@gmail.com

To Email

jameschristster@gmail.com

Subject

Testcases

Email Format

Text

Text

OUTPUT

envelopeTime	messageTime	messageSize
2652	852	6020

Editor

On form submission

HTTP Request

Basic LLM Chain

Send an Email

Groq Chat M

Execute workflow

Logs

Success in 10.458s

HTTP Request

Basic LLM Chain

Send an Email

Success in 9.715s

INPUT

text

```
Code Review - 'LearnDragAndDrop.java'
(Selenium + Java - simple drag-and-drop demo)
Parameter | Description (what was evaluated) | Score (out of 20)
-----|-----|-----
Code Readability | Naming follows Java conventions (driver, sourceFile, targetFile). <br> Indentation is consistent and the flow is easy to follow. <br> Missing package-level Javadoc, method-level comments and a brief
```

OUTPUT

accepted	rejected	ehlo
0 : jameschristster@gmail.com	[empty array]	0 : SIZE 35882577 1 : 8BITMIME